# Same use-after-free, two languages

A stale alias into the context map survives the erase that frees it (fault-injection catalogue, RQ2)

## C++: compiles cleanly

```
std::unordered_map<uint64_t, UeContext> ctxs;
ctxs[1001] = UeContext{1001, 0};
UeContext* p = &ctxs.at(1001); // alias
ctxs.erase(1001); // frees node p points to
p->messages_handled += 1;
^^ use-after-free (freed storage)
```

**→ Detected only at runtime, by AddressSanitizer, and only if a test exercises this path.**

## Safe Rust: does not compile

```
let mut ctxs: HashMap<u64, UeContext>
= HashMap::new();
ctxs.insert(1001, UeContext { .. });
let p = ctxs.get_mut(&1001).unwrap(); // borrow
ctxs.remove(&1001); // borrow still live
p.messages_handled += 1;

error[E0499]: cannot borrow `ctxs` as
mutable more than once at a time
```

**→ Rejected at compile time by the borrow checker. The defect cannot be written in the first place.**

### The difference is one of kind, not degree

C++: valid program (latent hazard) → runtime crash / corruption / tail-latency spike

Rust: build fails on every ordinary build → shipped code cannot contain this defect